

`ranges::to`: A function to convert any range to a container

Document #: P1206R1
Date: 2019-01-21
Project: Programming Language C++
Audience: LEWG
Reply-to: Corentin Jabot <corentin.jabot@gmail.com>
Eric Niebler <eric.niebler@gmail.com>
Casey Carter <casey@carter.net>

1 Abstract

We propose a function to copy or materialize any range (containers and views alike) to a container.

2 Revisions

Revision 1

- Split out the proposed constructors for string view and span into separate papers ([[P1391](#)] and [[P1394](#)] respectively)
- Use a function based approach rather than adding a constructor to standard containers, as it proved unworkable.

3 Quick Overview

We propose all the following syntaxes to be valid constructs

```
std::list<int> l;
std::map<int, int> m;

// copy a list to a vector of the same type
Same<std::vector<int>> auto a = ranges::to<std::vector<int>>(l);
//Specify an allocator
Same<std::vector<int, Alloc>> auto b = ranges::to<std::vector<int, Alloc>(l, alloc);
// copy a list to a vector of the same type, deducing value_type
Same<std::vector<int>> auto c = ranges::to<std::vector>(l);
// copy to a container of types ConvertibleTo
Same<std::vector<long>> auto d = ranges::to<std::vector<long>>(l);

//Supports converting associative container to sequence containers
Same<std::vector<std::pair<const int, int>>>
    auto f = ranges::to<vector<std::pair<const int, int>>>(m);

//Removing the const from the key by default
Same<std::vector<std::pair<int, int>>> auto e = ranges::to<vector>(m);

//Supports converting sequence containers to associative ones
Same<std::map<int, int>> auto g = f | ranges::to<map>();

//Pipe syntax
Same<std::vector<int>> auto g = l | ranges::view::take(42) | ranges::to<std::vector>();

//Pipe syntax with allocator
auto h = l | ranges::view::take(42) | ranges::to<std::vector>(alloc);

//The pipe syntax also support specifying the type and conversions
auto i = l | ranges::view::take(42) | ranges::to<std::vector<long>>();

//Pathentheses are optional for template
Same<std::vector<int>> auto j = l | ranges::view::take(42) | ranges::to<std::vector>;
//and types
auto k = l | ranges::view::take(42) | ranges::to<std::vector<long>>;
```

4 Tony tables

Before	After
<pre>std::list<int> lst = /*...*/; std::vector<int> vec {std::begin(lst), std::end(lst)};</pre>	<pre>std::vector<int> = lst ranges::to<std::vector>;</pre>
<pre>auto view = ranges::iota(42); vector < iter_value_t< iterator_t<decltype(view)> > > vec; if constexpr(SizedRanged<decltype(view)>) { vec.reserve(ranges::size(view)); } ranges::copy(view, std::back_inserter(vec));</pre>	<pre>auto vec = ranges::iota(42) ranges::to<std::vector>;</pre>
<pre>std::map<int, widget> map = get_widgets_map(); std::vector< typename decltype(map)::value_type > vec; vec.reserve(map.size()); ranges::move(map, std::back_inserter(vec));</pre>	<pre>auto vec = get_widgets_map() ranges::to<vector>;</pre>

5 Motivation

Most containers of the standard library provide a constructors taking a pair of iterators.

```
std::list<int> lst;
std::vector<int> vec{std::begin(lst), std::end(lst)};
//equivalent too
std::vector<int> vec;
std::copy(it, end, std::back_inserter(vec));
```

While, this feature is very useful, as converting from one container type to another is a frequent use-case, it can be greatly improved by taking full advantage of the notions and tools offered by ranges.

Indeed, given all containers are ranges (ie: an iterator-sentinel pair) the above example can be rewritten, without semantic as:

```
std::list<int> lst;
std::vector<int> vec = lst | ranges::to<std::vector>;
```

The above example is a common pattern as it is frequently preferable to copy the content of a `std::list` to a `std::vector` before feeding it an algorithm and then copying it back to a `std::vector`.

As all containers and views are ranges, it is logical they can themselves be easily built out of ranges.

5.1 View Materialization

The main motivation for this proposal is what is colloquially called *view materialization*. A view can generate its elements lazily (upon increment or decrement), such as the value at a given position of the sequence iterated over only exist transiently in memory if an iterator is pointing to that position. (Note: while all lazy ranges are views, not all views are lazy).

View materialization consists in committing all the elements of such view in memory by putting them into a container.

The following code iterates over the numbers 0 to 1023 but only one number actually exists in memory at any given time.

```
std::iota_view v{0, 1024};
for (auto i : v) {
    std::cout << i << ' ';
}
```

While this offers great performance and reduced memory footprint, it is often necessary to put the result of the transformation operated by the view into memory. The facilities provided by [\[P0896R3\]](#) allow to do that in the following way:

```
std::iota_view v{0, 1024};
std::vector<int> materialized;
std::copy(v, std::back_inserter(materialized));
```

This proposal allows rewriting the above snippet as:

```
auto materialized = std::iota_view{0, 1024} | std::ranges::to<std::vector>();
```

Perhaps the most important aspect of view materialization is that it allows simple code such as:

```
namespace std {
    split_view<std::string_view> split(std::string_view);
}
auto res = std::split("Splitting strings made easy")
    | std::ranges::to<std::vector>;
```

Indeed, a function such as `split` is notoriously hard to standardize ([\[P0540\]](#), [\[N3593\]](#)), because without lazy views and `std::string_view`, it has to allocate or expose an expert-friendly interface. The view materialization pattern further let the *caller* choose the best container and allocation strategy for their use case (or to never materialize the view should it not be necessary). And while it would not make sense for a standard-library function to split a string into a vector it would allocate, it's totally reasonable for most applications to do so.

This paper does not propose to standardize such `split` function - a `split_view` exist in [\[P0896R3\]](#), however, view materialization is something the SG-16 working group is interested in. Indeed, they

have considered APIs that could rely heavily on this idiom, as it has proven a natural way to handle the numerous ways to iterate over Unicode text. Similar ideas have been presented in [P1004].

```
auto sentences =
    text(blob)
    normalize<text::nfc> |
    graphemes_view |
    split<sentences> | ranges::to<std::vector<std::u8string>>;
```

6 Constructing views from ranges

Constructing standard views (`string_view` and `span`) from ranges is addressed in separate papers as the design space and the requirements are different:

- `string_view` : [P1391]
- `span` : [P1394]
- Work is being done to allow Ranges's iterators to be move only

As views are not containers, they are not constructible from `ranges::to`

7 Alternative designs

While we believe the range constructor based approach is the cleanest way to solve this problem, LEWG was interested in alternative design based on free functions

7.1 Range constructors

The original version of that paper proposed to add range constructors to all constructors. This proved to be unworkable because of `std::initializer_list`:

```
std::vector<int> foo = ....;
std::vector a{foo}; //constructs a std::vector<std::vector<int>>
std::vector b(foo); //would construct a std::vector<int>
```

8 Existing practices

8.1 Range V3

This proposal is based on the `to` (previously `(to_)` function offered by ranges v3.

```

auto vec = view::ints
| view::transform([](int i) {
    return i + 42;
})
| view::take(10)
| to<std::vector>;

```

8.2 Abseil

Abseil offer converting constructors with each of their view. As per their documentation:

One of the more useful features of the `StrSplit()` API is its ability to adapt its result set to the desired return type. `StrSplit()` returned collections may contain `std::string`, `absl::string_view`, or any object that can be explicitly created from an `absl::string_view`. This pattern works for all standard STL containers including `std::vector`, `std::list`, `std::deque`, `std::set`, `std::multiset`, `std::map`, and `std::multimap`, and even `std::pair`, which is not actually a container.

Because they can not modify existing containers, view materialization in Abseil is done by the mean of a conversion operator:

```

template<Container C>
operator C();

```

However, because it stands to reason to expect that there are many more views than containers and because conversions between containers are also useful, it is a more general solution to provide a solution that is not coupled with each individual view.

8.3 Previous work

[\[N3686\]](#) explores similar solutions and was discussed by LEWG long before the Ranges TS.

9 Future work

Eric Niebler suggested (and implemented in ranges v3) making the parenthesis `range::to<...>()` in optional, such that

```

auto v = view | range::to<vector<int>>;

```

Would be valid syntax. However, this is currently only be feasible for types `template (range::to<vector<int>>;)` and not `template template (range::to<vector>;)` which would require a core language tweak to make possible.

10 Proposed wording

We do not provide wording at this time, but this is what the interface would look like conceptually.

```
namespace ranges {

    //1
    template <Container C, Range R, typename...Arg>
    constexpr auto to(const R & r, Args...&) -> C;

    //2
    template <template <typename...> typename C,
              Range R, typename T = range_value_t<R>, typename... Args>
    constexpr auto to(const R & r, Args...&) -> C<T, Args...>;

    //3
    template <Container C, typename...Args>
    constexpr auto to(Args...&&) -> implementation-defined;

    //4
    template <template <typename...> typename C>
    constexpr auto to(Args...&&) -> implementation-defined;

    //5
    template <Range R>
    constexpr auto operator|(const R && r, implementation-defined{});

}
```

Functions 3 and 4 return an implementation defined object that can be passed to the pipe operator(5), and provide an implementation defined way of creating a container of the appropriate type.

`range::to` forwards the range to the container if it is constructible from it. If not, it tries to construct the container from the `begin(range)/end(range)` iterators pair. Otherwise, it falls-back to `range::copy`. This ensure the most efficient strategy is selected to perform the actual copy.

11 Implementation Experience

Implementations of this proposal are available on in the 1.0 branch of [\[RangeV3\]](#) and on [\[Compiler Explorer\]](#) (Incomplete cmcstl2-based prototype).

To make sure the parentheses are optional (`v| ranges::to<vector>;`) our implementations use a default constructed tag which dispatch through a function pointer. However, this have no runtime cost and doesn't suffer from the same issues LEWG had about `std::in_place_tag` because no actual indirection takes place. We believe being able to omit the parenthesis is necessary so `ranges::to` remains consistent with the syntax of views adaptors, and is otherwise a nice quality of life improvement in a facility which we expect to be used frequently.

12 Related Paper and future work

- [P1391] adds range and iterator constructor to `string_view`
- [P1394] adds range and iterator constructor to `span`
- [P1425] adds iterator constructors to `stack` and `queue`
- [P1419] Provide facilities to implementing `span` constructors more easily.

Future work is needed to allow constructing `std::array` from *tiny-ranges*.

13 Acknowledgements

We would like to thank the people who gave feedback on this paper, notably Christopher Di Bella, Arthur O'Dwyer, Barry Revzin and Tristan Brindle.

14 References

- [Compiler Explorer] <https://godbolt.org/z/m1I9NE>
- [RangeV3] Eric Niebler https://github.com/ericniebler/range-v3/blob/v1.0-beta/include/range/v3/to_container.hpp
- [P1391] Corentin Jabot *Range constructor for std::string_view*
<https://wg21.link/P1391>
- [P1394] Casey Carter, Corentin Jabot *Range constructor for std::span*
<https://wg21.link/P1394>
- [P1425] Corentin Jabot *Iterators pair constructors for stack and queue*
<https://wg21.link/P1425>
- [P1419] Casey Carter, Corentin Jabot *A SFINAE-friendly trait to determine the extent of statically sized containers*
<https://wg21.link/P1419>
- [P0896R3] Eric Niebler, Casey Carter, Christopher Di Bella *The One Range Ts Proposal*
<https://wg21.link/P0896>
- [P1004] Louis Dionne *Making std::vector constexpr*
<https://wg21.link/P1004>
- [P1004] Tom Honermann *Text_view: A C++ concepts and range based character encoding and code point enumeration library*
<https://wg21.link/P0244>

- [P0540] Laurent Navarro *A Proposal to Add split/join of string/string_view to the Standard Library*
<https://wg21.link/P0540>
- [N3593] Greg Miller *std::split(): An algorithm for splitting strings*
<https://wg21.link/N3593>
- [P1035] Christopher Di Bella *Input range adaptors*
<https://wg21.link/P1035>
- [Abseil] <https://abseil.io/docs/cpp/guides/strings>
- [N3686] Jeffrey Yasskin *[Ranges] Traversable arguments for container constructors and methods*
<https://wg21.link/n3686>
- [P1072R1] Chris Kennelly, Mark Zeren *Vector as allocation transfer device* <https://wg21.link/P1072>
- [1] [P0504R0] Jonathan Wakely *Revisiting in-place tag types for any/optional/variant* <https://wg21.link/P0504R0>